

计划任务

Linux **crontab** 是 Linux 系统中用于设置周期性被执行的指令的命令。

crond 命令每分钟会定期检查是否有要执行的工作，如果有要执行的工作便会自动执行该工作。

注意：新创建的 cron 任务，不会马上执行，至少要过 2 分钟后才可以，当然你可以重启 cron 来马上执行。

Linux 任务调度的工作主要分为以下两类：

- 1、**系统执行的工作：**系统周期性所要执行的工作，如备份系统数据、清理缓存
- 2、**个人执行的工作：**某个用户定期要做的工作，例如每隔 10 分钟检查邮件服务器是否有新信，这些工作可由每个用户自行设置

语法

```
crontab [ -u user ] file
```

或

```
crontab [ -u user ] { -l | -r | -e }
```

说明：

crontab 是用来让使用者在固定时间或固定间隔执行程序之用，换句话说，也就是类似使用者的时程表。

-u user 是指设定指定 user 的时程表，这个前提是你必须要有其权限(比如说是 root)才能够指定他人的时程表。如果不使用 -u user 的话，就是表示设定自己的时程表。

参数说明：

- `-e` : 执行文字编辑器来设定时程表, 内定的文字编辑器是 Vi/Vim, 如果你想用别的文字编辑器, 则请先设定 VISUAL 环境变数来指定使用那个文字编辑器(比如说 `setenv VISUAL joe`)
- `-r` : 删除目前的时程表
- `-l` : 列出目前的时程表

查看当前用户的 crontab 文件:

```
crontab -l
```

编辑当前用户的 crontab 文件:

```
crontab -e
```

删除当前用户的 crontab 文件:

```
crontab -r
```

时间格式

时间格式如下:

```
f1 f2 f3 f4 f5 program
```

- 其中 f1 是表示分钟, f2 表示小时, f3 表示一个月份中的第几日, f4 表示月份, f5 表示一个星期中的第几天。program 表示要执行的程序
- 当 f1 为 `*` 时表示每分钟都要执行 program, f2 为 `*` 时表示每小时都要执行程序, 以此类推
- 当 f1 为 `a-b` 时表示从第 a 分钟到第 b 分钟这段时间内要执行, f2 为 `a-b` 时表示从第 a 到第 b 小时都要执行, 以此类推
- 当 f1 为 `*/n` 时表示每 n 分钟个时间间隔执行一次, f2 为 `*/n` 表示每 n 小时个时间间隔执行一次, 以此类推
- 当 f1 为 `a, b, c,...` 时表示第 a, b, c,... 分钟要执行, f2 为 `a, b, c,...` 时表示第 a, b, c...个小时要执行, 以此类推

51**5

echo "hello world" >> /tmp/hello.txt

-

-

-

-

-

|

|

|

|

|

|

|

|

|

+-----

星期中星期几 (0 - 6) (星期天 为0)

|

|

|

+

月份 (1 - 12)

|

|

+

一个月中的第几天 (1 - 31)

|

+

小时 (0 - 23)

+

分钟 (0 - 59)

时间表达式	说明
0 * * * *	每小时的整点执行一次
*/10 * * * *	每10分钟执行一次
30 3 * * *	每天凌晨3:30执行
0 9 * * 1	每周一上午9:00执行
0 0 1 * *	每月第1天的00:00执行
0 0 * * 0	每周日的00:00执行
0 9-18 * * 1-5	周一至周五的早上9点到下午6点，每小时整点执行
0,30 12,18 * * *	每天12:00, 12:30, 18:00, 18:30执行
0 4 1,15 * *	每月1号和15号的凌晨4:00执行

应用举例

每周日 12点清理30天前的日志文件：

```
0 0 * * 0 find /var/log -name "*.log" -mtime +30 -exec rm {} \;
```

SSH协议

Secure Shell(SSH) 是由 IETF(The Internet Engineering Task Force) 制定的建立在应用层基础上的安全网络协议。它是专为远程登录会话(甚至可以用Windows远程登录Linux服务器进行文件互传)和其他网络服务提供安全性的协议，可有效弥补网络中的漏洞。通过SSH，可以把所有传输的数据进行加密，也能够防止DNS欺骗和IP欺骗。

SSH提供两种级别的验证方法：

第一种级别（基于口令的安全验证）：只要你知道自己帐号和口令，就可以登录到远程主机。

第二种级别（基于密钥的安全验证）：你必须为自己创建一对密钥，并把公钥放在需要访问的服务器上。

SSH两种级别的远程登录

一、口令登录

```
ssh 客户端用户名@服务器ip地址
```

```
# 范例
```

```
ssh root@172.16.5.5
```

```
ssh root@172.16.5.5 -p 2222
```

如果是第一次登录远程主机，系统会给出下面提示：

```
$ ssh root@172.16.5.5 -p 22
The authenticity of host '172.16.5.5 (172.16.5.5)' can't be
established.
ED25519 key fingerprint is
SHA256:YW44gGuuyry2qNqUe4Nkdv4y9H0aQ3P9w+DqnKj0McI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
yes
Warning: Permanently added '172.16.5.5' (ED25519) to the list of
known hosts.
```

输入yes即可。这时系统会提示远程主机被添加到已知主机列表。

已知主机列表存放在 `~/.ssh/known_hosts` 文件中

二、公钥登录

在本机生成密钥对

使用 `ssh-keygen` 命令生成密钥对

```
$ ssh-keygen -t rsa
```

会在 `~/.ssh` 目录下创建 2 个文件

```
[root@localhost ~]# cd ~/.ssh/
[root@localhost .ssh]# ll
total 8
-rw----- 1 root root 2610 Sep  3 21:39 id_rsa
-rw-r--r-- 1 root root  580 Sep  3 21:39 id_rsa.pub
```

 私钥

 公钥

将公钥复制到远程主机中

使用 `ssh-copy-id` 命令将公钥复制到远程主机。

`ssh-copy-id` 会将公钥写到远程主机的 `~/.ssh/authorized_keys` 文件中

```
$ ssh-copy-id root@172.16.5.5
```

远程复制

从远程机器复制文件到本地目录

```
# 文件  
$ scp root@172.16.5.5:/root/set_docker.sh /root/  
  
# 目录  
$ scp -r root@172.16.5.5:/opt/1panel /tmp/
```

上传本地文件到远程机器指定目录

```
# 文件  
$ scp test.1 root@172.16.5.5:/home/  
  
# 目录  
$ scp -r study root@172.16.5.5:/home/
```